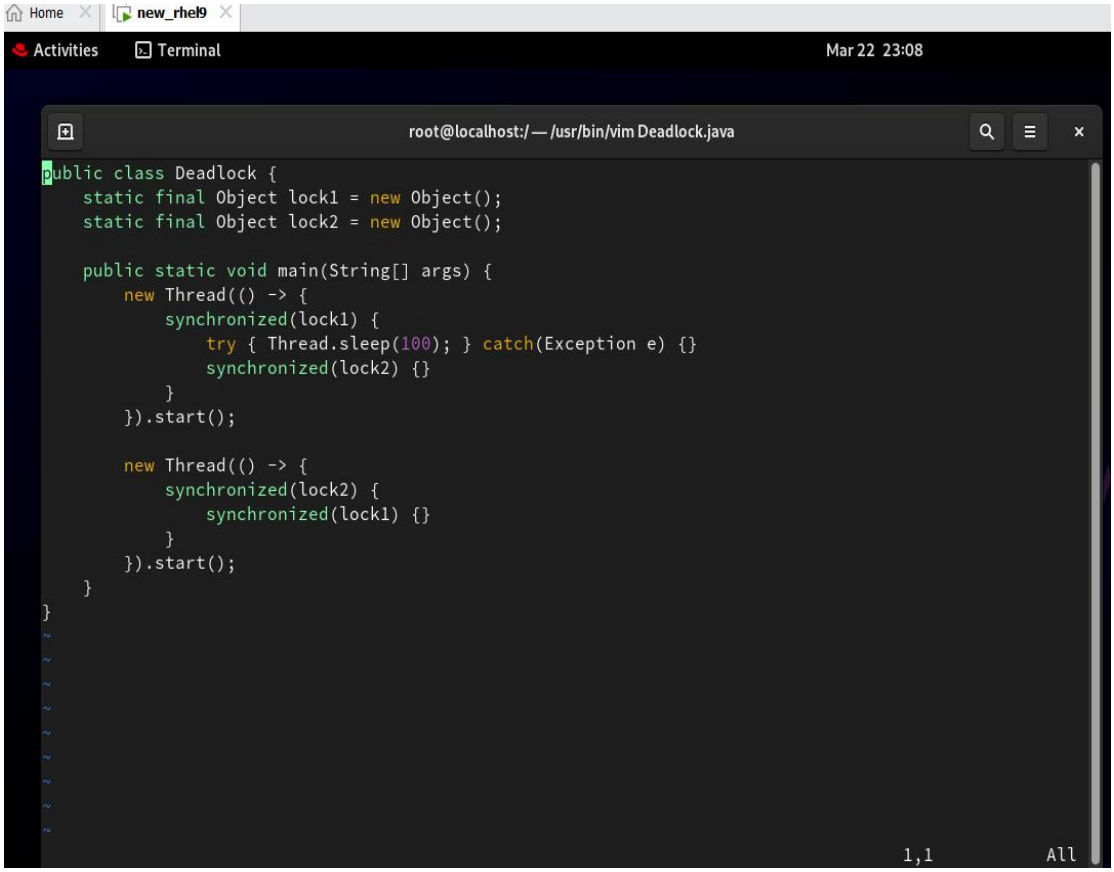
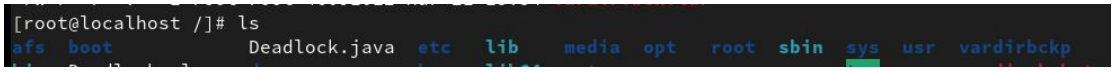
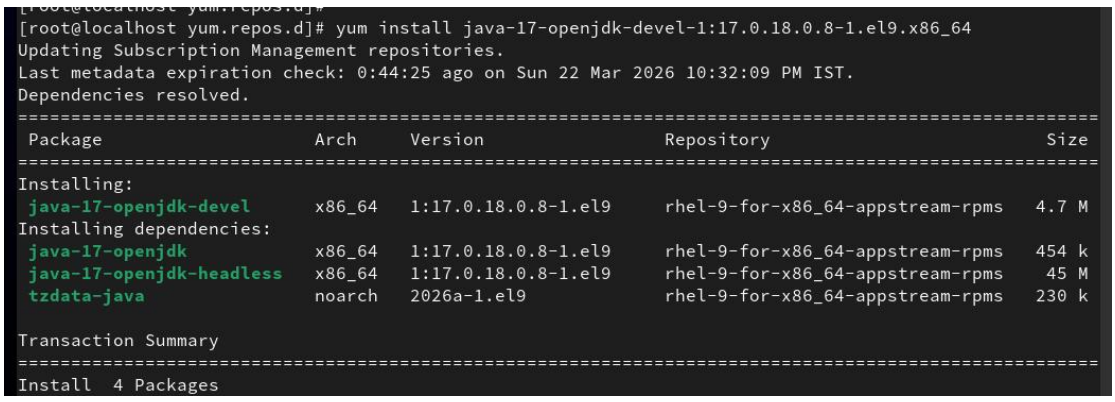
Step1. I created a simple Java program/process(Deadlock.java)just for practice, so I can learn how to take a Java thread dump.



step2. First, I verified that the Java file (`Deadlock.java`) is created in the system.



step3. To run a Java program, Java (JDK) must be installed on the server.So, I installed Java using the package manager (`yum install java-17-openjdk-devel`). now u can run comand like javac,jstack,java,jps.



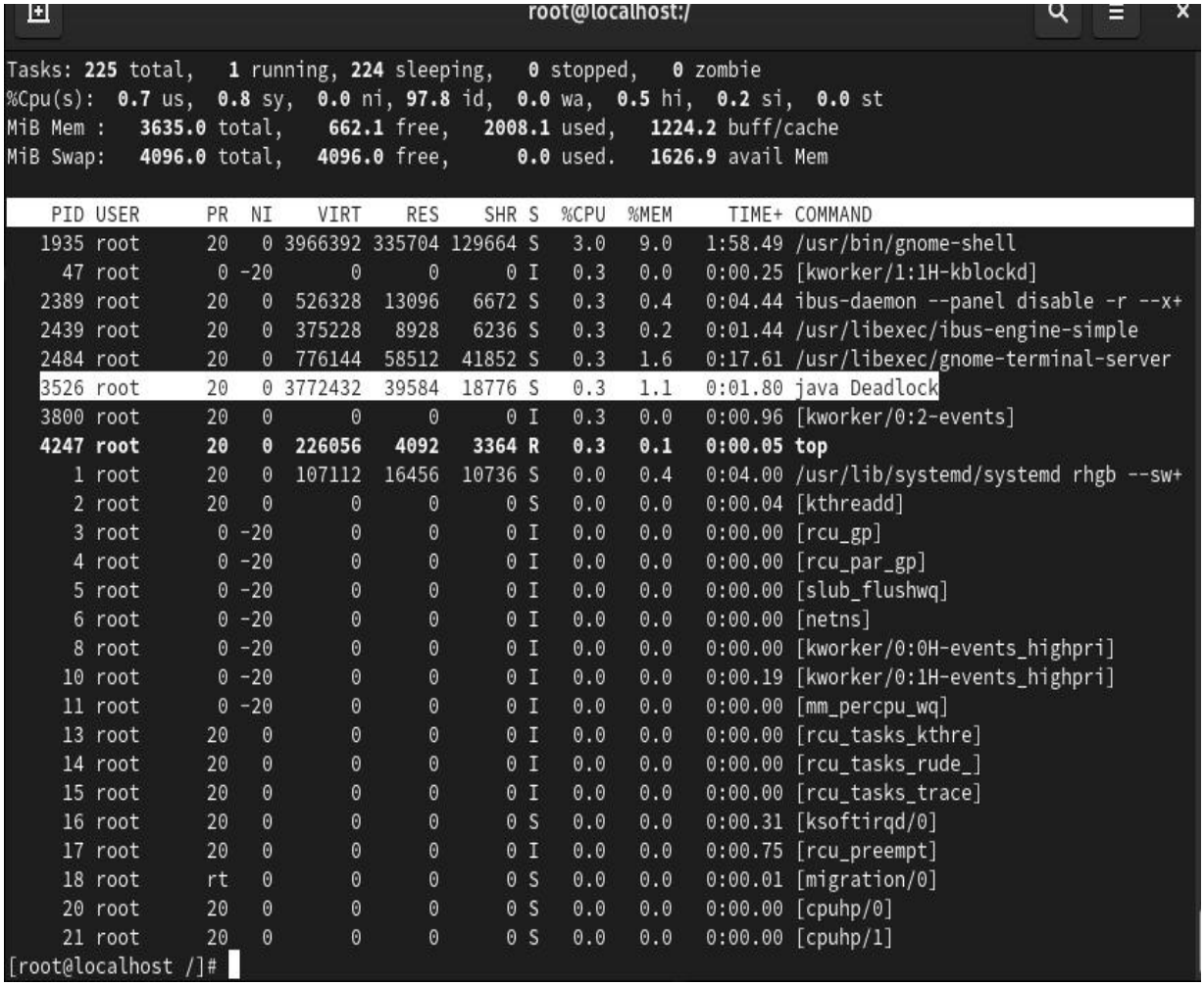
Step4. After installing Java, I compiled the program using: javac Deadlock. java this command creates a bytecode file (`Deadlock.class`).

Then, I ran the program using: java Deadlock This starts the Java application.

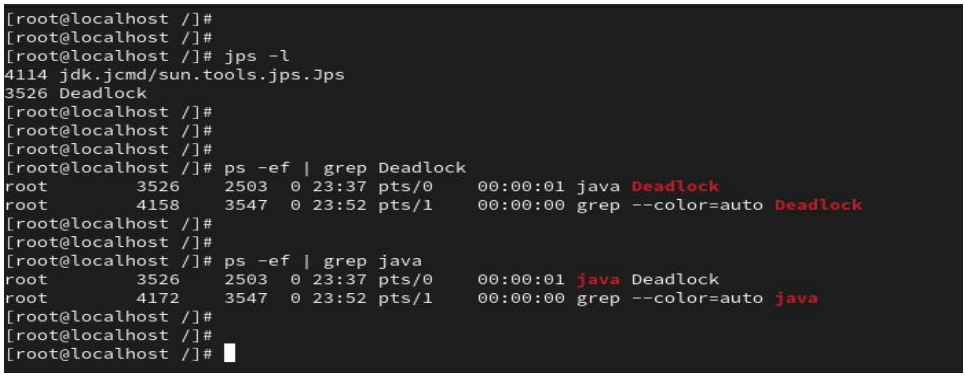
```
[root@localhost /]#
[root@localhost /]# javac Deadlock.java
[root@localhost /]# java Deadlock
```

Step5. Verify Application is Running and Find PID

After running the Java program, I verified whether the application is running.First, I used the `top` command to check running processes and confirmed that the `java Deadlock` process is active.

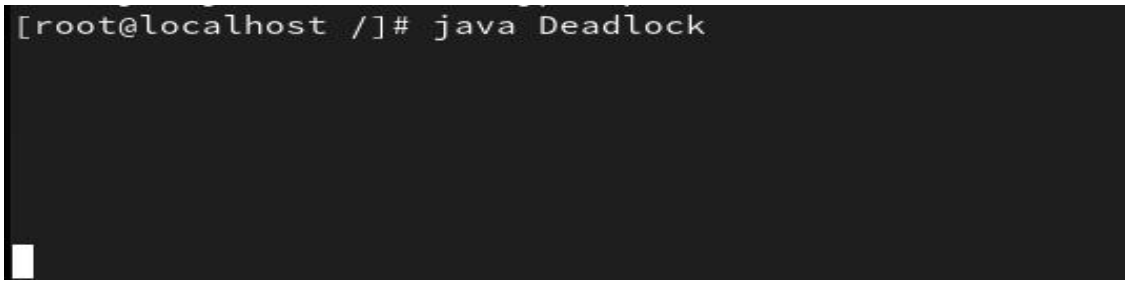


Step 6. Then, I used the `jps -l` command to list Java processes, where I could see the `Deadlock` program along with its PID. I also used the `ps -ef | grep Deadlock` command to find the process manually.Additionally, I verified using `ps -ef | grep java` to check all Java-related processes.



Step7. The Java program is running, but after some time it gets stuck and stops working.Even though the process is active, it is not performing any operation.This indicates there may be an issue,

So I decided to take a thread dump to analyze the problem.



step8. Before taking the thread dump, I checked the TIDs (Thread IDs) of the Java process using its PID by running the command top -H -p <PID>

I observed that none of the threads were consuming high CPU or memory.Even though resource usage was normal, the application was still stuck.This indicates that the issue is not related to CPU or memory, but possibly due to a deadlock or thread waiting condition.

```
[root@localhost /]# top -H -p 3526

top - 23:58:53 up 41 min,  2 users,  load average: 1.14, 0.67, 0.43
Threads:  21 total,   0 running,  21 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.3 us,  0.3 sy,  0.0 ni, 98.8 id,  0.0 wa,  0.3 hi,  0.2 si,  0.0 st
MiB Mem :   3635.0 total,    652.8 free,   2017.3 used,   1224.2 buff/cache
MiB Swap:   4096.0 total,   4096.0 free,      0.0 used.   1617.7 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   3526 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.00 java
   3527 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.06 java
   3528 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.00 GC Thread#0
   3529 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.00 G1 Main Marker
   3530 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.00 G1 Conc#0
   3531 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.00 G1 Refine#0
   3532 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.31 G1 Service
   3533 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.05 VM Thread
   3534 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.00 Reference Handl
   3535 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.00 Finalizer
   3536 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.00 Signal Dispatch
   3537 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.00 Service Thread
   3538 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.21 Monitor Deflati
   3539 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.01 C2 CompilerThre
   3540 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.00 C1 CompilerThre
   3541 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.00 Sweeper thread
   3542 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.00 Notification Th
   3543 root      20   0 3772432  39672  18776 S   0.0   1.1   0:01.29 VM Periodic Tas
   3544 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.00 Common-Cleaner
   3545 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.05 Thread-0
   3546 root      20   0 3772432  39672  18776 S   0.0   1.1   0:00.04 Thread-1
```

step9. so, I took the thread dump using the `jstack` command with the PID of the Java process.After running the command, I verified that the thread dump file was successfully created.

This dump file will be used to analyze the issue in the Java application.

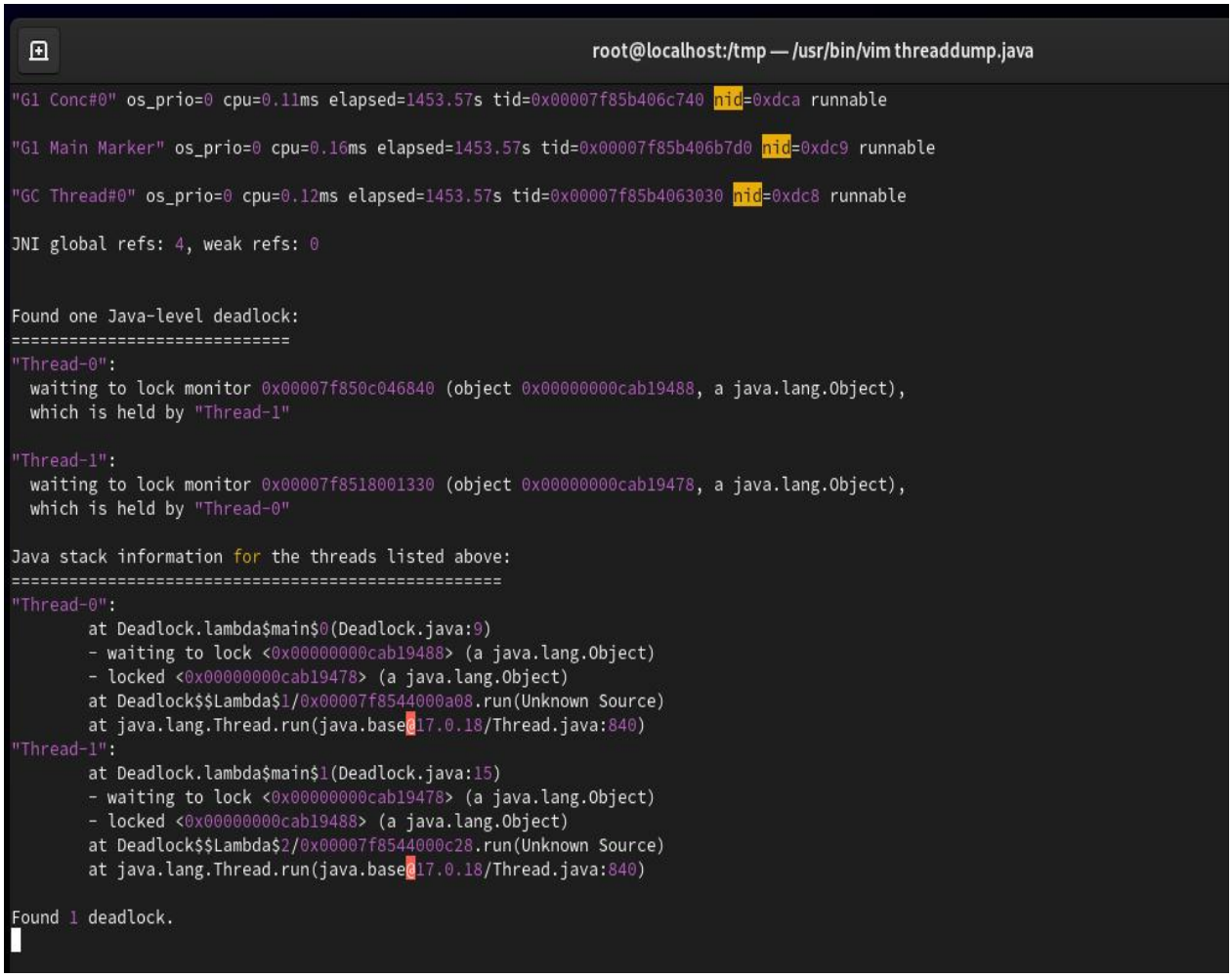
step10. Analyze Thread Dump

After taking the thread dump, I opened the dump file to analyze it.The thread dump shows the state of all threads in the Java application.

In the dump file, I can see that some threads are in a waiting/blocked state.It clearly shows that one thread is waiting to acquire a lock held by another thread, and the second thread is waiting for the first thread.

The dump also explicitly mentions "Found one Java-level deadlock", which confirms the issue.It provides details of both threads, including which locks they are holding and which locks they are waiting for.

From this thread dump, I confirmed that the application is stuck due to a deadlock.



step11 . After analyzing the thread dump file, I identified the problematic threads using their nid (native thread ID).The thread dump shows `nid` in hexadecimal format (e.g., `0xdd9`, `0xdda`).

To match these with system-level thread IDs (TIDs), I converted the `nid` values from hexadecimal to decimal using the `printf` command.

step12 .After converting the `nid` values to TIDs, I matched them with the threads using the `top -H -p 3526` command.I identified the problematic threads in the system.Both threads were in a waiting state and not consuming CPU.From the analysis, it is clear that both threads are waiting for each other's resources.